Nombre: Landon Alexander Bautista Hernandez Karla Judith Baez Villanueva Alexa Daiana Amaya Torres	**Matrícula**: 2101662 2107930 2100425
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 24	**Actividad**: ACT-F03. Revisión con SOLID
Fecha: 16/10/2026	
Bibliografía (Formato APA) Archivo “03_Sazón_Urbano.docx” proporcionado en la plataforma TEAMS	

Revisión con SOLID

Antes de cerrar con los diagramas, pasamos el diseño por cada principio SOLID y lo contrastamos con lo que ya teníamos escrito. Buscamos tres cosas: clases que cargaran con demasiado, dependencias que no hacían falta y reglas que nos obligaran a abrir código que ya funciona. Algunas decisiones salieron bien y otras las ajustamos. Las que decidimos no tocar quedan registradas como riesgo.

Tabla de revisión SOLID

Principio	Qué revisamos	Qué encontramos	Decisión
S Responsabilidad única	Servicios, handlers y aggregates	Pedido, Pago y Entrega tienen su propio servicio, y los métodos multipaso solo derivan la solicitud al handler correspondiente. El aggregate Inventario junta existencias, mermas y alertas	Se mantiene la separación de servicios. Inventario queda como riesgo aceptado
O Abierto/cerrado	Reglas que podrían crecer: pagos, promociones y lealtad	Promoción ya calcula su propio descuento. En Pago, el estado inicial dependía del método con condiciones encadenadas. Los puntos de lealtad estaban fijos en "1 por cada $10"	Se corrigen Pago y lealtad
L Sustitución de Liskov	Repositorios en memoria frente a relacionales	El contrato cubría los repositorios con id, pero InventarioRepository.obtener() no tenía definido qué pasa en el primer arranque	Se amplía el contrato
I Segregación de interfaces	Qué necesita realmente cada consumidor	Los handlers de pedido y promoción solo preguntan si hay stock, y las queries usan interfaces de lectura	Se mantiene
D Inversión de dependencias	Dirección de las dependencias	Dominio y aplicación solo conocen interfaces, y el Pedido recibe la disponibilidad ya consultada en lugar de pedirla al inventario	Se mantiene

Detalle de lo que ajustamos

Pago y métodos de cobro. Efectivo exige monto recibido y calcula cambio, tarjeta se registra como aprobada sin guardar datos de la tarjeta, y transferencia queda pendiente hasta que el Encargado la verifica. Si todo eso vivía en una cadena de condiciones dentro de Pago, un método nuevo (por ejemplo QR) obligaría a editar esa

clase. Ahora cada método es una variante que define su estado inicial y sus validaciones, y Pago no cambia cuando aparece otra.

Puntos de lealtad. La Historia 18 y el RF-18 hablan de un programa que el gerente configura, pero la regla estaba escrita dentro de Cliente. Pasó a ser una política de lealtad vigente que Cliente solo aplica, sin conocer el valor.

Contrato del inventario. Como obtener() no recibe id, definimos que siempre devuelve un inventario (vacío si aún no existe) o lanza un error de negocio. Así la versión en memoria y la relacional se comportan igual desde el primer uso.

Riesgos que dejamos documentados

- **Pedido concentra mucho comportamiento.** Maneja estado, líneas, promoción y cancelación. No lo partimos porque dividirlo rompería la consistencia que protege como aggregate. Lo acotamos con value objects y con AplicadorDePromocion.
- **Inventario es grande.** Con más de 150 ingredientes, cargarlo completo puede pesar. Lo mantenemos unido porque cancelar un pedido debe devolver ingredientes o registrar la merma en la misma operación. Si el módulo crece, el primer candidato a separarse es Merma.

Cambios que arrastra a otras etapas

Las correcciones de Pago, lealtad y contrato de inventario modifican las fichas de Pago y Cliente (Etapa 3) y el contrato común (Etapa 5), y deben reflejarse también en el diagrama de clases y en la bitácora.